

Week 3 Notebook 1 - ML Revision with a Realistic Regression Problem

Problem: estimate food delivery time

Last week we learned that machine learning changes **parameters** from examples. Today we will quickly review that idea using a realistic prediction problem.

We will use one input first:

- **input:** distance in kilometers
- **target:** actual delivery time in minutes
- **model:** `predicted_time = w * distance + b`

This is still machine learning, but it is **not** a neural network yet.

Part 0 - Import only what we need

We will use `matplotlib` only because visualizing the data makes the idea easier to understand.

```
import matplotlib.pyplot as plt
```

Part 1 - Look at the data before coding the model

Each pair is `(distance_km, delivery_time_minutes)`. The data is intentionally a little noisy, because real measurements are rarely perfect.

Prediction before running: Should the points roughly form a line? Why or why not?

```
DATA = [  
    (1.0, 8.5),  
    (2.0, 11.4),  
    (3.0, 15.0),  
    (4.0, 18.0),  
    (5.0, 21.0),  
    (6.0, 24.8),  
]  
  
for distance, minutes in DATA:  
    print(distance, "km ->", minutes, "min")  
  
# Visualize the examples.  
distances = [example[0] for example in DATA]  
times = [example[1] for example in DATA]
```

```
plt.figure()
plt.scatter(distances, times)
plt.xlabel("distance (km)")
plt.ylabel("delivery time (min)")
plt.title("Delivery time examples")
plt.show()
```

Part 2 - Define a prediction rule

Our first model is a line:

```
predicted_time = w * distance + b
```

- **w** is the slope: extra minutes per km
- **b** is the baseline time: minutes even before travel distance

Prediction before coding: If **w = 3** and **b = 5**, what should the model predict for a 4 km delivery?

```
def predict(distance, w, b):
    """Predict delivery time from distance using a linear model."""
    # TODO 1: return w * distance + b
    pass

print(predict(4.0, w=3.0, b=5.0))
```

Part 3 - Measure one mistake

A prediction by itself is not learning. The model needs feedback.

We measure:

```
error = prediction - actual
```

If error is positive, the prediction was too high. If error is negative, the prediction was too low.

```
distance = 5.0
actual = 21.0
w = 2.0
b = 5.0

prediction = predict(distance, w, b)
error = prediction - actual

print("prediction:", prediction)
print("actual:", actual)
print("error:", error)
```

```
# TODO 2: In a comment, write whether the model predicted too high or too low.
```

Part 4 - Measure many mistakes with MSE

For many examples, we use mean squared error:

```
MSE = average((prediction - actual)^2)
```

This gives one score for how wrong the model is on the dataset.

```
def mean_squared_error(data, w, b):  
    """Return the average squared error over all examples."""  
    total = 0  
    for distance, actual in data:  
        prediction = predict(distance, w, b)  
        error = prediction - actual  
        # TODO 3: add the squared error to total  
        # total += ...  
    return total / len(data)  
  
print(mean_squared_error(DATA, w=2.0, b=5.0))
```

Part 5 - Act like the learning algorithm

Before automatic training, let's manually try several slopes.

Prediction before running: Which value of w do you expect to give the smallest loss?

```
candidate_ws = [1.5, 2.0, 2.5, 3.0, 3.5]  
b = 5.0  
  
for w in candidate_ws:  
    loss = mean_squared_error(DATA, w, b)  
    print("w =", w, "loss =", round(loss, 3))
```

Part 6 - Visualize candidate models

A number like loss is useful, but a picture helps us understand what is happening.

```
def plot_model(data, w, b):  
    distances = [example[0] for example in data]  
    times = [example[1] for example in data]  
  
    x_line = [0, 7]  
    y_line = [predict(x, w, b) for x in x_line]
```

```
plt.figure()
plt.scatter(distances, times)
plt.plot(x_line, y_line)
plt.xlabel("distance (km)")
plt.ylabel("delivery time (min)")
plt.title(f"Model: time = {w} * distance + {b}")
plt.show()
```

TODO 4: Try two different values of w and compare the plots.

```
plot_model(DATA, w=2.0, b=5.0)
plot_model(DATA, w=3.0, b=5.0)
```

Part 7 - Concept check: what did the model learn?

Answer in markdown or comments:

1. What does w mean in this delivery problem?
2. What does b mean?
3. Why is distance alone useful but incomplete?
4. Is this neural-network code yet? Why or why not?

TODO 5: Write short answers here.

```
# 1.
# 2.
# 3.
# 4.
```

Bridge to Notebook 2

Distance is useful, but real delivery time also depends on traffic, rain, demand, and restaurant preparation time.

A model can use multiple inputs:

```
score = w1 * distance + w2 * traffic + w3 * rain + b
```

That weighted-sum idea is the beginning of a neuron.